# ICT912 Programming

**Topic 2**

Variables, Data Types and Expressions

CIHE Australia

# Resources

**Prescribed Texts**

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

**Recommended Readings**

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

* Slides taken from Cengage resources

CENGAGE

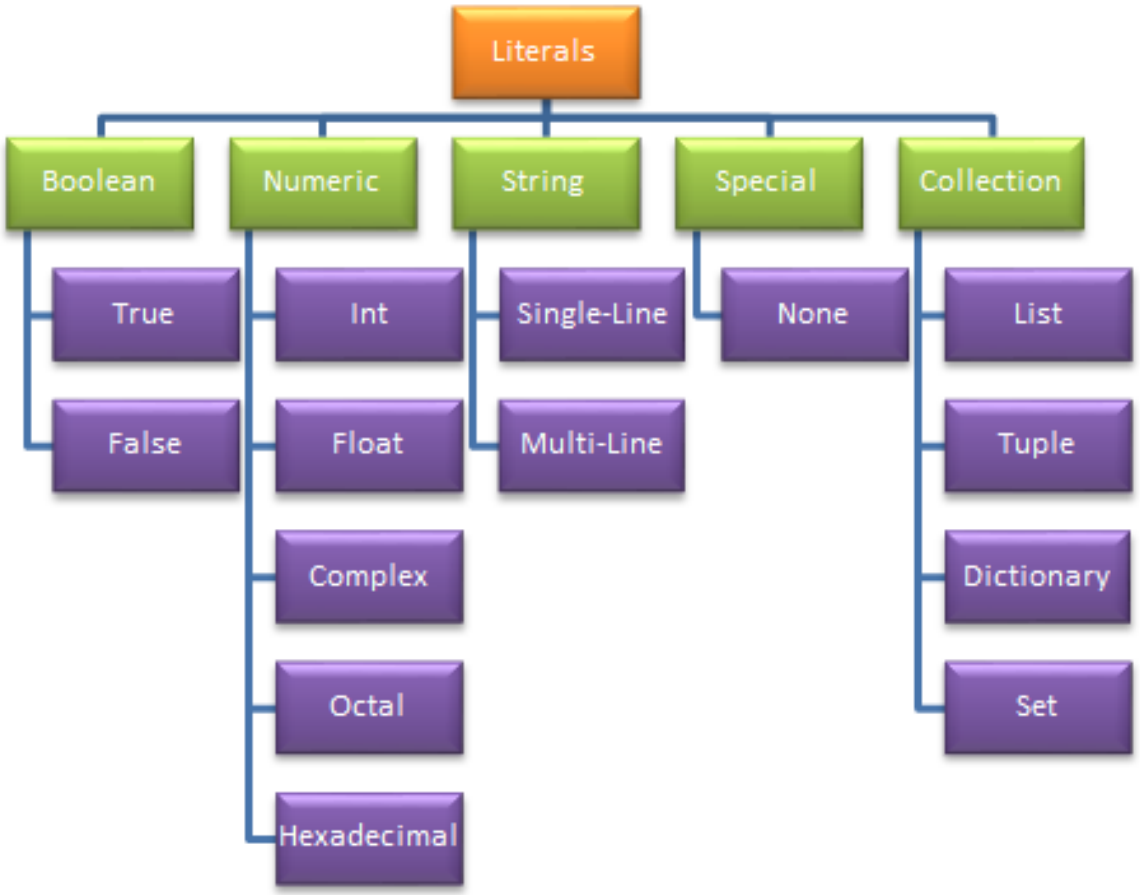
# Lesson Learned from Week 1:

- Understand the need for programming
- Explain job prospects and the role of a programmer in the industry
- Develop problem-solving skills and understand the problem-solving approach
- Understand algorithms, flowcharts and their significance in programming
- Learn why Python is widely used and its advantages over other languages
- Understand fundamental programming terminologies

# Week 2: Lesson Learning Outcomes

- Declare, initialize and manipulate variables
- Explain significance of primitive data types
  - Demonstrate operational skills with mathematical operators
- Identify and manipulate character data and string data
  - Use built-in functions to manipulate strings
- Explain meanings of coercion and typecasting in the context of data types

# Literals




- Literals are constant values that appear directly in the program

- Types of Literals:

| Literals | Description |
|---|---|
| Boolean | Represents two expressions, **True** or **False** |
| Numeric | Defines numbers. **Int**, **Float**, **Complex**, **Octal**, H**exadecimal** |
| String | Sequence of letters, numbers, special symbols. |
| Special | Known as **None**. Field has not been created. |
| Collection | Made up of different primitive datatypes. **List**, **Set**, **Tuple**, and **Dictionary** |

# Examples

## Numeric Literals

**Decimal literals**

```
75
196
3046
```

**Hex literals**

```
0x4b
0xC4
0xBE6
```

**Binary literals**

```
0b1001011
0b11000100
0b101111100110
```

**Floating-point literals**

```
75.3
.196
3046.00
```

**Exponent literals**

```
7.53e+1
1.96e-1
3.05e+3
```

## Character and String Literals

**Character literals**

```
"a"
'T'
"#"
```

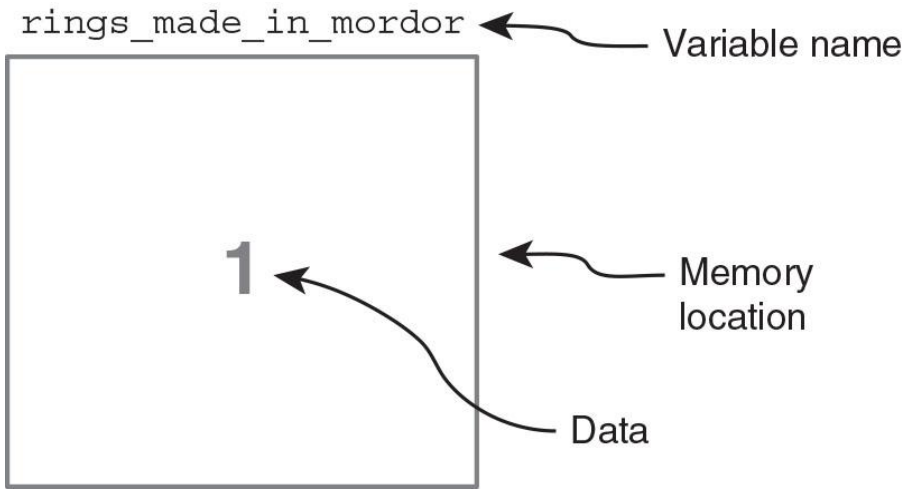
**String literals**

```
"cat"
'A full sentence.'
"1932"
```

# Variables

- A named memory location that temporarily holds text or a numeric value

- Three important characteristics
  - A name
  - A corresponding memory location where data can be stored
  - Changeable (mutable) data
    - *Vary* + *able* that means its value can be changed



# Variable Naming Conventions

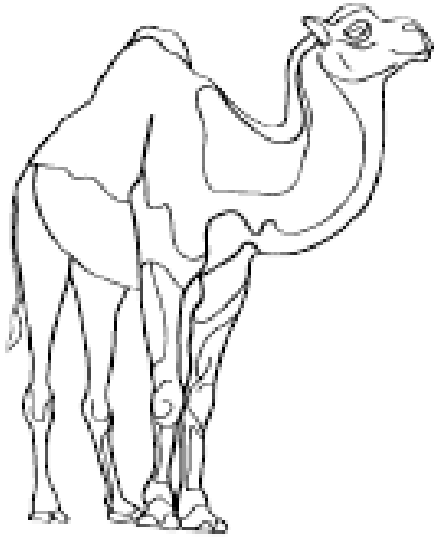
- General conventions:
  - In Python, you cannot use the name of one of the language's built-in functions, such as print
  - Names should be descriptive
  - Python variable names must start with a letter or an underscore

- Two most prevalent conventions for combining words for variable names:
  - **Camel case** begins with a lowercase letter, but each subsequent word is capitalized
  - **Snake case** uses all lowercase and separates words with an underscore

- Python typically uses camel case for variable names

# Examples

NadzeyaShanchuk/ Shutterstock.com



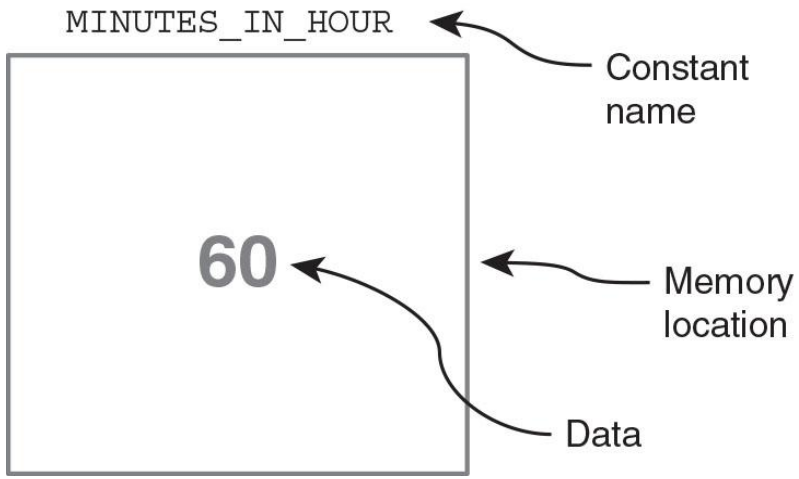
Margarita L/Shutterstock.com

| Snake case | Camel case |
|---|---|
| elven_rings | elvenRings |
| made_in_mordor | madeInMordor |
| hobbit | hobbit |
| multiples_of_2 | multiplesOf2 |

# Constants

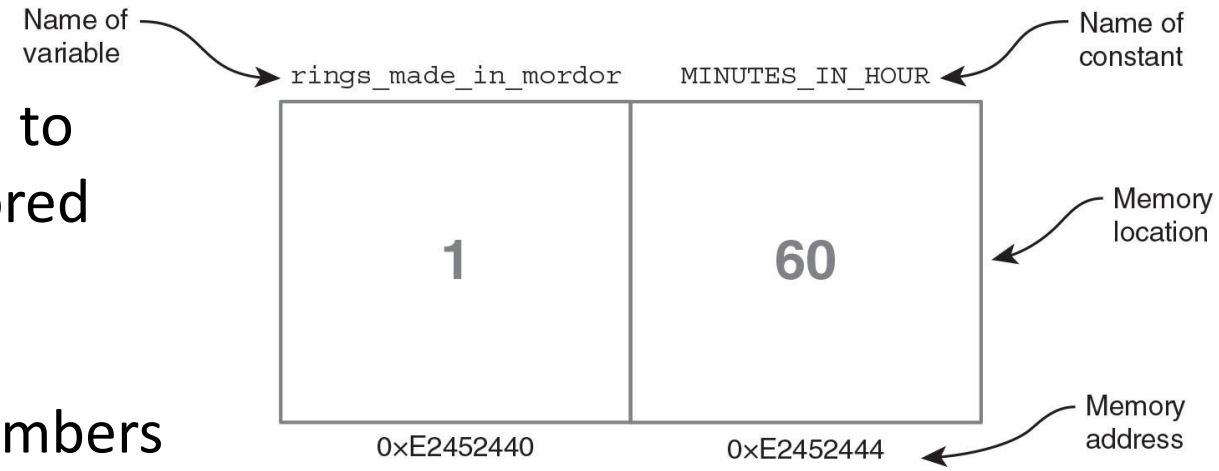
- A named memory location that holds data, which is used but not changed by the statements in a program

- Examples:
  - MINUTES_IN_HOUR = 60
    MAXIMUM_USERS = 1024
    OHIO_ABBREVIATION = "OH"



# Variables and Constants

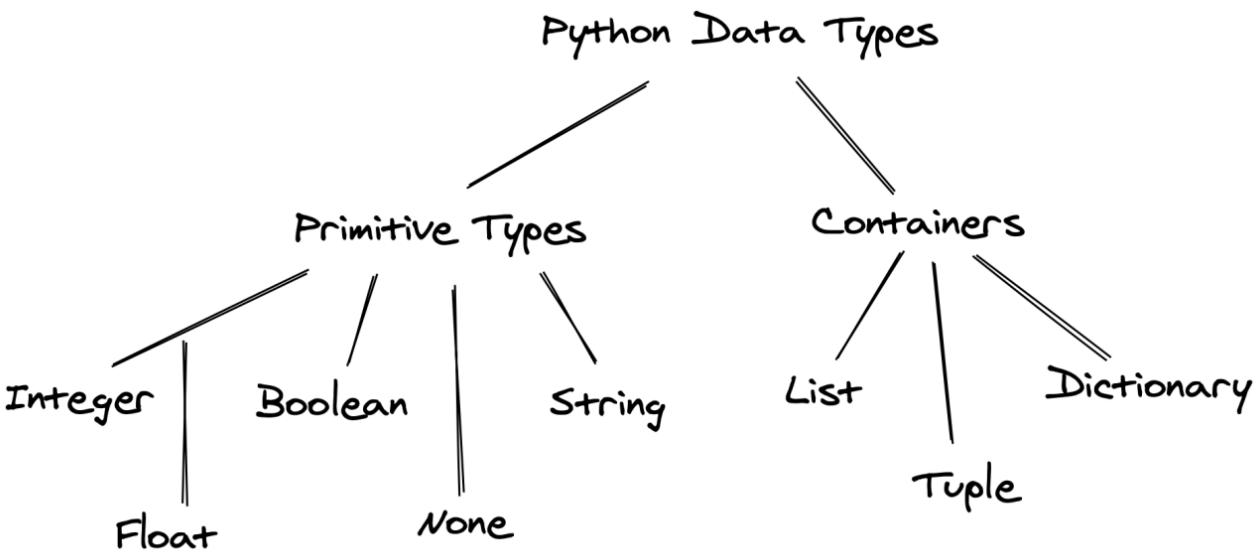
- The memory connection
  - Both variables and constants correspond to memory locations where data can be stored
  - These memory locations have unique addresses, expressed in binary or hex numbers
  - When you specify the name for a variable or constant, your programming language ties it to a memory location (the address) where you can put data



# Primitive Data Types

- Data types: a way of categorizing data

- Primitive data types are built into a programming language
- Programming language knows how to
  - allocate memory for storing these data types
  - Manipulate that type of data



# Primitive Data Types

| Primitive Data Type | Python Code | Description | Examples |
|---|---|---|---|
| Integer | var = 5 | Whole number without any decimal places | 1<br>128<br>3,056<br>-2 |
| Floating point | var = 5.0 | Number that includes decimal places | 12.99<br>0.26<br>-3.56 |
| Boolean | var = False | Logical value of True or False | True<br>False |
| Valueless | var = None | No value | No real-world equivalent examples |
| String | var = “Cake” | Sequence of characters (letters, numbers, spaces or special symbols) enclosed within single or double quotes | “John Doe”<br>‘123 Main Street’<br>‘abc@xyz.com’ |

# Composite Data Types

- Not built into programming language, but can be constructed from primitive data types

- Ex.: first song on Hamilton soundtrack is 3:56 minutes long
  - The value is not an integer, floating-point number or other primitive data type.
  - Data that represents time would be handled as a composite data type

# Assignment Statements

- Declaring variables
  - Specifying a name that is assigned to a memory location
  - Python requires you to assign a value to all variables with an initialization statement while declaring them

- Initialising variables
  - Specify a name and a value

| Dynamically typed | Type inference | Statically typed |
|---|---|---|
| a programming language (such as Python) that allows a variable to take on any type of data during program execution | process in which a language deduces the data type based on the presence or absence of a decimal point and quote marks | a programming language that requires you to store only one type of data in a variable |

# Role of Initialisation

- Specifying starting values when known
  - Ex.: total_rings = 3
- Specifying the initial value of a repetition counter (typically 0 or 1)
  - Ex.: counter = 0
- Creating a null variable (one that has no value), which is assigned to the keyword None in Python
  - Ex.: ring_location = None

# Mathematical Expressions

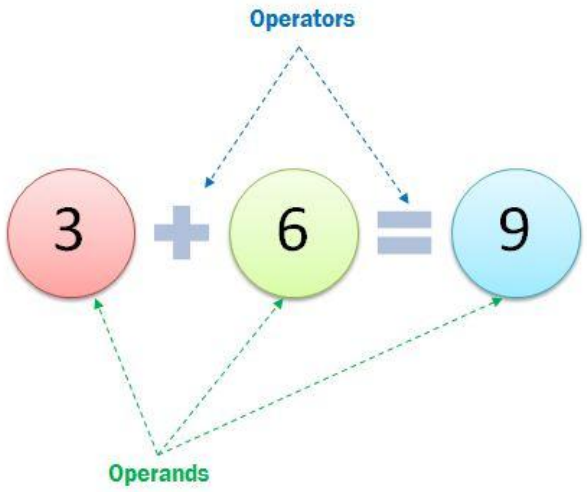
- Expression:
  - A programming statement that has a value and usually includes one or more arithmetic operators and one or more operands

```
x = 2 * 3 + 1
```

expression

- A symbol that instructs the compiler to perform a specific action is an **operator**

- The objects or values on which operators act are known as **operands**



# Arithmetic Operators

| Operator | Operation | Example | Result |
|---|---|---|---|
| + | Addition | a+b | The arithmetic sum of a and b |
| - | Subtraction | a-b | b subtracted from a |
| * | Multiplication | a * b | The product of a and b |
| / | Division | a / b | The quotient of a divided by b, expressed as a float |
| // | Integer division (floor division) | a // b | The quotient of a divided by b, rounded to the next smallest whole number |
| % | Modulo division | a % b | The remainder of a divided by b |
| ** | Exponentiation | a**b | a raised to the power of b |

```
>>> a = 5
>>> b = 2

>>> a + b
7
>>> a - b
3
>>> a * b
10
>>> a / b
2.5
>>> a % b
1
>>> a // b
2
>>> a**b
25
```

# Order of Operations

- Specifies the sequence in which to perform arithmetic operations
  - Do the math in parentheses first
  - Carry out exponentiation and roots
  - Perform multiplication, floating-point division, integer division, and modulo as they appear from left to right
  - Execute addition and subtraction as they appear from left to right
- Parentheses can be added around the operation you wish to perform first to alter the order of operations

# Example

```
without_parenths = 3 + 2 * 5

with_parenths = (3 + 2) * 5

print("Without parentheses:", without_parenths)

print("With parentheses:", with_parenths)
```

OUTPUT:

```
Without parentheses: 13

With parentheses: 25
```

# Input and Output

```
# using input() to take user input
num = input('Enter a number: ')

print('You Entered:', num)
```

- Input to a variable
  - Take user input from the keyboard using **input()**
  - To collect data from a user:
    - Initialise a variable to hold the user's input
    - Supply a prompt that explains what you want the user to enter
    - Collect the data entered by the user in the initialized variable
  - Python allows you to include initialization, a display and an input statement on the same line

- Output from a variable
  - Display output to the console using **print()**
  - When variables are sent as output to a display, the output is the value of the variable, not the variable name

# Formatting Output

- Formatting parameters:
  - Type: integer, floating point, decimal, binary, hexadecimal, character, or E notation
  - Precision: places to be displayed after the decimal point
  - Width: number of spaces allocated to the output (e.g. for outputting table columns)
  - Alignment: left or right within a specified width

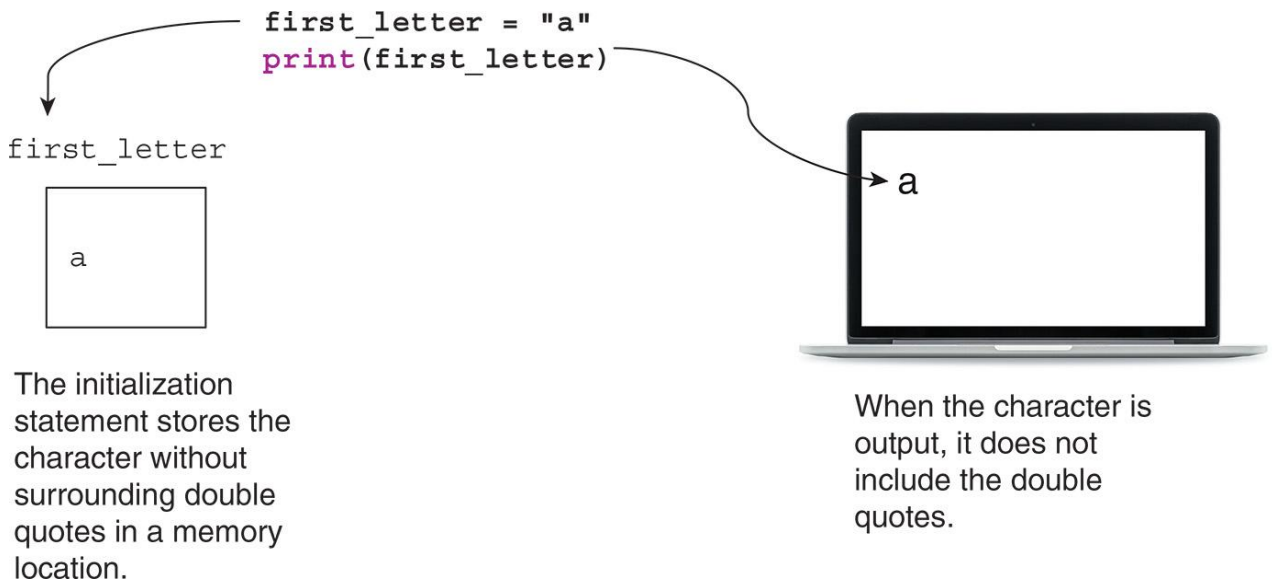
- Syntax for the Python format() method
  - `"Your lucky numbers are {:d} and {:d}.".format(42, 7.3)`
  - Formatting parameters are inserted inside the curly braces

# Formatting Parameters

| Parameter | Letter or Symbol | Example Code | Output |
|---|---|---|---|
| Type | d for decimal integers<br>f for floating-point | "{:d}".format(100)<br>"{:f}".format(15.3) | 100<br>15.300000 |
| Precision | Decimal point followed by the precision; only works for float numbers | "{:.3}".format(15.3) | 15.3 |
| Width | A number representing the number of characters for the total width | "{:10}".format(2) | _____2 |
| Left-aligned | < | "{:<10}".format(2) | 2_____ |
| Centered | ^ | "{:^10}".format(2) | ____2_____ |
| Right-aligned | > | "{:>10}".format(2) | _____2 |

# Character Data Type

- Classification used for variables that hold a single letter of the alphabet, a numeral from 0 to 9, or a symbol
- Character data is stored in ASCII format




Various-Everythings/Shutterstock.com

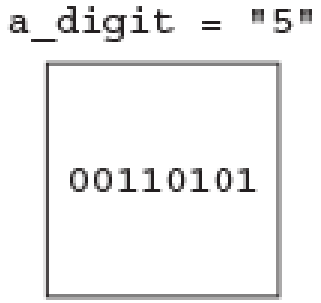
- Ex.: “a” is represented by eight-bit ASCII code 01100001, which is decimal 97
- Each of the uppercase letters, lowercase letters, numerals, and symbols has a unique ASCII value

# Digits

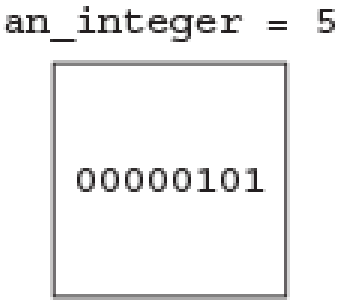
- The double quotes around strings composed of the digits 0 to 9 indicate they are strings, not integers

```
first_digit = "2"
second digit = "5"
print(first_digit + second_digit)
```

The correct answer is "25" !




The digit 5 is stored as its ASCII value.




The integer 5 is stored as a binary number

# Character Manipulation

| Character Function/Method | Example | Data | Result |
|---|---|---|---|
| Check if the character is a letter of the alphabet | initial.isalpha()<br>choice.isalpha() | initial = "a"<br>choice = "2" | True<br>False |
| Check if the character is a digit | initial.isdigit()<br>choice.isdigit() | initial = "a"<br>choice = "2" | False<br>True |
| Change to uppercase | initial.upper() | initial = "a" | A |
| Change to lowercase | initial.lower() | initial = "A" | a |

# String Data Type

- String:
  - A sequence of alphanumeric characters
- The string data type is used for working with variables containing words, phrases, sentences, and other text

- String literals are enclosed in double quotes (“ ”) for initialization statements, assignment statements, and prompts.

# Examples

- Strings can be initialized, input and displayed

| | |
|---|---|
| your_name = “” | Initializes **your_name** as an empty string. |
| message = “likes alphabet soup.” | Initializes a variable containing a string. Quotes are required in the program statement. |
| print(“What is your name?”) | Displays a prompt containing a string. |
| your_name = input() | Collects a user-input string such as Skylar Jones. No quotes are needed when the user enters the string. |
| print(your_name, message) | Outputs the string contents of **your_name** followed by the string stored in the variable called message separated by a space. |

```
your_name = input("What is your name? ")
message = "likes alphabet soup."
print(your_name, message)
```

OUTPUT:

```
What's your name?
Skylar Jones [Enter]
Skylar Jones likes alphabet soup.
```

User input

# String Indexes

- Index: a number that indicates a character's position in a string

- The first character in the string is referenced by index 0, the next character is index 1, and so on.

| Memory Location | 0x0075 | 0x0076 | 0x0077 | 0x0078 | 0x0079 | 0x007A | 0x007B | 0x007C | 0x007D | 0x007E |
|---|---|---|---|---|---|---|---|---|---|---|
| Data | C | a | m | p | b | e | l | l | ' | s |
| Index | [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] | [9] |

- Each character in a string is stored in a memory location referenced by an index.

# Examples

```
company_name = "Campbell's"
letter = company_name[0]
print(letter)
```

OUTPUT:

```
c
```

```
company_name = "Campbell's"
letter = company_name[2]
print(letter)
```

OUTPUT:

```
m
```

# String Manipulation

| String Function/Method | Example | Data | Result |
|---|---|---|---|
| Return a string's length as an integer | `len(alphabet)` | `alphabet = "ABCDEFGH"` | 8 |
| Returns the number of times a specified value occurs in a string | `str.count("a")` | `str = "banana"` | 3 |
| Returns True if the string ends or starts with the specified value | `str.endswith("bet")`<br>`str.startswith("cat")` | `str = "alphabet"` | True<br>False |
| Returns the first index of the specified value or -1 if it can't be found | `str.find("p")`<br>`str.find("t")` | `str = "apple"` | 1<br>-1 |
| Capitalizes the first letter of each word in the string | `str.title()` | `str = "hello world!"` | Hello World! |
| Convert every character in the string to lowercase or uppercase | `str.lower()`<br>`str.upper()` | `str = "Soup"` | soup<br>SOUP |
| Find a substring and store it in a variable | `sub_str = alphabet[2:5]` | `alphabet = "ABCDEFGH"` | CDE |

# Concatenation

- Concatenation:
  - the process of chaining two or more values in sequence
- The + sign is the concatenation operator

- You can concatenate one or more variables and store the combined string in a new variable

```
company_name = "Campbell's"
soup_name = "Alphabet Soup"
product_name = company_name + " " + soup_name
print(product_name)
```

```
Campbell's Alphabet Soup
```

```
prompt the user for their first and last names
read the first and last names
display a message to the last name, first name format
```

EXPECTED OUTPUT:

```
What is your first name? Philip [Enter]
What is your last name? Parker [Enter]
Hello, Philip
You are filed as Parker, Philip
```

```
first_name = input("What is your first name? ")
last_name = input("What is your last name? ")
print("Hello,", first_name)
print("You are filed as", last_name + ", " + first_name)
```

OUTPUT:

```
Hello, Philip
You are filed as Parker, Philip
```

# Escape Characters

- Escape characters allow special characters to be used in strings without breaking the syntax.
- They start with a backslash (\).

| Escape Sequence | Meaning | Escape Sequence | Meaning |
|---|---|---|---|
| \n | New line | \” | Double quote (“) |
| \t | Tab space | \r | Carriage return |
| \\ | Backslash (\) | \b | Backspace |
| \’ | Single quote (‘) | \f | From feed |

- What do you expect as the output from this code?

```
review = "This song's kind of \n \"SOUPY!\""
print(review)
```

```
This song's kind of
 "SOUPY!"
```

# Coercion and Typecasting

**Coercion**

- Implicit Type Conversion
- Python automatically converts one data type to another.
- E.g.

```
result = 10 + 2.5  # Integer is converted to float
print(result)  # Output: 12.5
```

**Typecasting**

- Explicit Type Conversion
- Manually converting a value from one type to another
- Using functions like `int()`, `float()`, `str()`
- E.g.

```
num = "100"
converted = int(num)  # String to Integer
print(converted + 50)  # Output: 150
```

# Next Week – What's Coming Up?

- Describe the purpose of control structures
- Identify and define relational operators
- Explain if-then, if-then-else, nested-if, else-if and switch-case decision structures
  - Illustrate using flowcharts
- Identify logical operators in conditional statements

CIHE

CIHE Australia